

Algorithm 945: modred—A Parallelized Model Reduction Library

BRANDT A. BELSON, JONATHAN H. TU, and CLARENCE W. ROWLEY,
Princeton University

We describe a new parallelized Python library for model reduction, modal analysis, and system identification of large systems and datasets. Our library, called *modred*, handles a wide range of problems and any data format.

The *modred* library contains implementations of the Proper Orthogonal Decomposition (POD), balanced POD (BPOD) Petrov-Galerkin projection, and a more efficient variant of the Dynamic Mode Decomposition (DMD). The library contains two implementations of these algorithms, each with its own advantages. One is for smaller and simpler datasets, requires minimal knowledge to use, and follows a common matrix-based formulation. The second, for larger and more complicated datasets, preserves the abstraction of vectors as elements of a vector space and, as a result, allows the library to work with arbitrary data formats and eases distributed memory parallelization. We also include implementations of the Eigensystem Realization Algorithm (ERA), and Observer/Kalman Filter Identification (OKID). These methods are typically not computationally demanding and are not parallelized. The library is designed to be easy to use, with an object-oriented design, and includes comprehensive automated tests. In almost all cases, parallelization is done internally so that scripts that use the parallelized classes can be run in serial or in parallel without any modifications.

Categories and Subject Descriptors: G.1.3 [Numerical Analysis]: Numerical Linear Algebra; G.4 [Mathematical Software]

General Terms: Algorithms, Experimentation, Performance

Additional Key Words and Phrases: Model reduction, system identification, Python, Proper Orthogonal Decomposition, Balanced Proper Orthogonal Decomposition, Dynamic Mode Decomposition, Koopman modes, Observer/Kalman Filter Identification, Eigensystem Realization Algorithm

ACM Reference Format:

Brandt A. Belson, Jonathan H. Tu, and Clarence W. Rowley. 2014. Algorithm 945: *modred*—A parallelized model reduction library. *ACM Trans. Math. Softw.* 40, 4, Article 30 (June 2014), 23 pages.
DOI: <http://dx.doi.org/10.1145/2616912>

1. INTRODUCTION

As computational power and storage have increased, so too has the size of datasets. Fortunately, the high-dimensional systems that generate large datasets are often dominated by low-dimensional dynamics. By isolating these dynamics, we can approximate high-dimensional systems using low-dimensional models, which can be cheaply simulated, saving significant resources. Further, such models can also be useful for finding control laws which can then be applied to the original, high-dimensional system. For example, one might be interested in finding the most energetic modes of

This work was supported by the National Science Foundation under grant CMMI-0932928 and a GRFP and by the Air Force Office of Scientific Research under grant FA9550-09-1-0257.

Authors' present addresses: B. Belson, 31435 Waters Way, Lewes, DE 19958; email: belson17@gmail.com; J. H. Tu, University of California, Berkeley, Department of Electrical Engineering and Computer Science, Cory Hall 558, Berkeley, CA 94720-1774; email: jhtu@berkeley.edu; C. W. Rowley, D-Wing Engineering Quadrangle, Princeton University, Princeton, NJ 08540; email: cwrowley@princeton.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

2014 Copyright is held by the Owner/Author. Publication rights licensed to ACM. 0098-3500/2014/06-ART30 \$15.00

DOI: <http://dx.doi.org/10.1145/2616912>

a vibrating structure so as to dampen vibrations, but these modes can be hard to see from raw data [Azeez and Vakakis 2000]. The implementations of model-reduction methods tend to be for a specific problem or are only applicable to relatively small systems and datasets. A need has grown for model reduction in many fields, and the high-performance multipurpose library described here, *modred*, is designed to fill that need.

One method we include is the Proper Orthogonal Decomposition (POD). From an arbitrary set of data, POD yields a set of orthonormal modes which represent the dominant directions, in the L_2 sense, in the original data. The modes are ranked and typically few are necessary to accurately reproduce the original dataset. The method originated as a way to analyze random data, and is known by various alternative names, such as principal component analysis (PCA) [Pearson 1901] and the Karhunen-Loève (KL) decomposition [Loève 1955]. In a wide range of fields, it has proven to be a useful tool for identifying important coherent structures and for developing reduced-order models via Petrov-Galerkin projection. POD has a long history of applications in fluid dynamics [Lumley 1967; Holmes et al. 2011; Sirovich 1987; Aubry et al. 1988]. POD has also been used for studying vibrations, oscillations, and microelectromechanical systems [Liang et al. 2002; Azeez and Vakakis 2000]. In geophysics, POD modes are often called empirical orthogonal functions (EOF), and are used to observe strong climate patterns [Holton 2004]. POD is also used in image processing, chemical engineering, control theory, and dynamical systems.

Balanced POD (BPOD) is a variant of POD that has desirable properties for constructing reduced-order models of large, linear input-output systems. It produces two sets of bi-orthogonal modes, and projecting the original linear system onto these modes results in a model that accurately reproduces the input-output dynamics of the original system [Rowley 2005]. BPOD is designed to approximate *balanced truncation* [Moore 1981], a model reduction technique commonly used in control theory that has good a priori error bounds. BPOD is particularly useful when the original system is too large for standard control design techniques (or model reduction techniques) to be applied: the algorithm is tractable even when the state dimension is large, and control design can then be done using the reduced-order model. BPOD is applicable to any system of linear ordinary differential equations (ODEs), including discretizations of partial differential equations (PDEs). The method has been applied to several fluid dynamics systems for control purposes [Ilak and Rowley 2008; Ahuja and Rowley 2010; Dergham et al. 2011]. Our library includes methods to find BPOD modes and also to project a linear system onto these modes to find a reduced-order model.

Another modal decomposition technique included in *modred* is the Dynamic Mode Decomposition (DMD) [Schmid 2010], which identifies modes that oscillate at fixed frequencies. For systems with oscillatory dynamics, these structures may be more indicative of the flow physics than the most energetic (POD) modes, which may contain mixed frequency content. DMD has been applied to both computational and experimental data, for instance to a jet in crossflow and the wake behind a flexible membrane [Rowley et al. 2009; Schmid 2010]. Connections between the DMD and Koopman operator theory suggest that DMD modes may be useful for capturing the behavior of nonlinear systems [Rowley et al. 2009].

Finally, *modred* also includes system identification techniques: the Eigensystem Realization Algorithm (ERA) and Observer/Kalman Filter Identification (OKID). ERA takes a series of Markov parameters (defined by (16)) and from them produces a reduced-order input-output model. The resulting models are theoretically equivalent to those produced by BPOD, but computing them does not require computing adjoint simulations or finding two sets of modes, making ERA computationally cheaper by orders of magnitude for large systems [Juang and Pappa 1985; Ma et al. 2011]. Its

primary application is control design for high-order systems. It also can be applied to experimental data since it does not require data from an adjoint system.

OKID estimates Markov parameters from arbitrary, noisy, input-output data from any linear system [Juang et al. 1991]. This is especially useful for experimental data since it is often difficult to directly measure the Markov parameters. The output from OKID is suitable for input to ERA to form a reduced-order model from experimental data [Hsieh et al. 2006; Brunton et al. 2013].

All of these methods are applicable in a broad range of fields, and the goal of the modred library is to provide a flexible, efficient, and scalable package that implements them. The library is written in Python because Python is easy to use, freely available, and can easily interface with existing code, including compiled C/C++ and Fortran. Users can supply functions that interact with their data (e.g., reading from files), making modred compatible with any type of data. modred is object-oriented and has been carefully designed, making it easy to use and easy to extend to new algorithms in the future. Automated tests are included for all of its functionality, and the library has already been effectively used on several datasets. The code is parallelized with MPI (using the mpi4py library) and demonstrates excellent scalability up to hundreds of processes. A key reason for this is the abstract vector-space approach described in Section 2.

The remainder of the article is organized as follows. Section 2 gives an overview of the modal decompositions POD, BPOD, and DMD, using an approach which emphasizes vector-space operations and abstract inner products. Short code samples are also presented. In Section 3, we summarize how to find reduced-order models using Petrov-Galerkin projection, ERA, and OKID and provide more code samples. Section 4 describes the software design principles we employ. Section 5 explains the parallelization of the modal decompositions.

2. MODAL DECOMPOSITIONS

In this section, we describe three algorithms for decomposing a given dataset into its dominant components. The inputs to each algorithm are *vectors*, for instance from a set $\{\mathbf{x}_i \in V\}$, where V is a vector space, and the outputs of each are *modes*, for instance, from a set $\{\boldsymbol{\varphi}_j \in V\}$, usually computed as a linear combination of the $\mathbf{x}_i$, as

$$\boldsymbol{\varphi}_j = \sum_i [\mathbf{T}]_{i,j} \mathbf{x}_i, \quad j = 1, \dots, r, \quad (1)$$

where $[\mathbf{T}]_{i,j}$ denotes the element in row i and column j of the matrix $\mathbf{T}$. Note that by vector, we do not necessarily mean a one-dimensional array—a vector could be a one-dimensional array, multidimensional array, or something else. When $\mathbf{x}_i$ represents a frame of data taken at an instant of time, as is very often the case, it is referred to as a *snapshot*. Here, we occasionally refer to input vectors as snapshots even when they are not necessarily sampled in time. The resulting modes most often serve as a low-order basis to approximate a generic vector $\mathbf{x} \in V$ via linear combinations,

$$\mathbf{x} \approx \sum_{j=1}^r q_j \boldsymbol{\varphi}_j, \quad (2)$$

where q_j are scalar coefficients that depend on $\mathbf{x}$.

The three algorithms we discuss produce modes with different properties: Proper Orthogonal Decomposition (POD) extracts the most energetic structures from the data, and the resulting modes are orthogonal; Balanced POD (BPOD) is based on input-output dynamics and produces two sets of modes that are bi-orthogonal; and

Dynamic Mode Decomposition (DMD) separates structures by frequency content, producing modes that are not necessarily orthogonal. Since both the snapshots and the modes are elements of a vector space V , we refer to them as vectors.

While the modal decompositions produce different modes, the algorithms to compute the modes share the same general procedure. First, the vector space V is established by defining an inner product, vector addition, and scalar multiplication. (Of course, these operations need to satisfy the properties of a vector space.) Then a decomposition (e.g., a singular value decomposition) is done to compute the matrix of coefficients $\mathbf{T}$ in (1). The last step is to form the modes via Eq. (1).

We will show two different approaches to all of the algorithms. The first is based on matrix multiplications and is only suited for smaller and simpler datasets because it requires that all of the vectors be flattened into one-dimensional arrays (mapping V to $\mathbb{R}^n$ or $\mathbb{C}^n$) and be stacked into large data matrices. When the datasets are small enough to be stacked into matrices, this matrix multiplication is computationally efficient thanks to highly optimized matrix libraries.

However, sometimes all of the data is too large to fit entirely into a single node's memory simultaneously. For these cases, we adopt a second approach that is based solely on vector space operations and therefore does not require vectors to be flattened into one-dimensional arrays or coerced into any particular data structure. Instead, it requires only vector addition, scalar multiplication, and inner products between vectors. Since each operation requires only one or two vectors in memory simultaneously, this approach is ideal for larger and more complicated datasets, and is parallelized for distributed memory architectures via MPI.

In the upcoming sections, we describe the steps for finding POD modes using the matrix multiplication and vector space approaches, highlighting the differences and showing code samples. We also explain the steps for finding BPOD and DMD modes, omitting some matrix multiplication steps that easily follow from the description of POD.

2.1. POD

Mathematically, the Proper Orthogonal Decomposition (POD) of a dataset $\{\mathbf{x}_i \in V \mid i = 1, \dots, m_{\mathbf{x}}\}$ arises when solving for the projection $\mathbf{P}_r$ of rank r that minimizes the error

$$\text{error} = \sum_{i=1}^{m_{\mathbf{x}}} \|\mathbf{x}_i - \mathbf{P}_r \mathbf{x}_i\|^2. \quad (3)$$

The operation $\|\cdot\|$ is the induced norm from the inner product $\langle \cdot, \cdot \rangle$ on V (linear in the second argument, conjugate linear in the first). The projection in Eq. (3) is written as

$$\mathbf{P}_r \mathbf{x}_i = \sum_{j=1}^r \langle \boldsymbol{\varphi}_j, \mathbf{x}_i \rangle \boldsymbol{\varphi}_j, \quad (4)$$

where $\boldsymbol{\varphi}_i \in V$ is the orthonormal basis of rank r such that the projection onto them minimizes the error in Eq. (3). The elements of this basis are called the *POD modes*, and we describe two closely related and mathematically equivalent ways of deriving and computing them in the upcoming paragraphs.

The first way uses matrix representations of the vectors. It is common for a generic vector $\mathbf{x} \in V$ to be represented as a two- or three-dimensional array of data, and if V has dimension n , a vector can be “flattened” into a column vector $\hat{\mathbf{x}} \in \mathbb{R}^n$ (or $\mathbb{C}^n$). (More precisely, the n components of $\hat{\mathbf{x}}$ are the coordinates of $\mathbf{x}$ with respect to some chosen basis of V .) The standard inner product in V is given by $\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = \mathbf{x}_1^* \mathbf{x}_2$, where $*$ denotes the conjugate transpose. However, the inner product may be weighted,

in which case $\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = \hat{\mathbf{x}}_1^* \mathbf{W} \hat{\mathbf{x}}_2$, where $\mathbf{W}$ is a Hermitian positive definite matrix. The POD modes are orthonormal with respect to this inner product and are ranked by their importance in the minimization of the error given by (3), in which the norm is given by $\|\mathbf{x}\|^2 = \langle \mathbf{x}, \mathbf{x} \rangle$. For many physical applications, the norm is chosen so that minimizing the error corresponds physically to optimally capturing the kinetic energy.

We summarize the steps to find the modes via matrix multiplication in this article. Here we show the method of snapshots [Sirovich 1987] and solve the resulting $m_{\mathbf{x}} \times m_{\mathbf{x}}$ symmetric eigenvalue problem. The method of snapshots formulation is well suited for large data because the eigenvalue problem does not depend on the dimension of V . For further details, see Sirovich [1987] and Holmes et al. [2011].

- (1) Collect and store vectors $\mathbf{x}_i \in V$, for $i = 1, \dots, m_{\mathbf{x}}$.
- (2) Flatten vectors into columns $\hat{\mathbf{x}}_i \in \mathbb{R}^n$ (or $\mathbb{C}^n$), where n is the dimension of V , and stack them in $\mathbf{X}$,

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ \hat{\mathbf{x}}_1 & \hat{\mathbf{x}}_2 & \dots & \hat{\mathbf{x}}_{m_{\mathbf{x}}} \\ | & | & & | \end{bmatrix} \quad (5)$$

so $\mathbf{X}$ is $n \times m_{\mathbf{x}}$.

- (3) Compute the $m_{\mathbf{x}} \times m_{\mathbf{x}}$ correlation matrix via $\mathbf{H} = \mathbf{X}^* \mathbf{W} \mathbf{X}$.
- (4) Compute the eigenvalues and eigenvectors of $\mathbf{H}$, writing $\mathbf{H} \mathbf{U} = \mathbf{U} \mathbf{\Sigma}$, where $\mathbf{\Sigma}$ is diagonal and real, and $\mathbf{U}$ is orthogonal (or unitary), since $\mathbf{H}$ is symmetric (or Hermitian). Sort the eigenvalues (and corresponding eigenvectors) by largest magnitude.
- (5) Select the number of modes to keep, r . Truncate the matrices, keeping the first r columns of $\mathbf{U}$ to obtain $\mathbf{U}_r$, and the first r rows and columns of $\mathbf{\Sigma}$ to obtain $\mathbf{\Sigma}_r$.
- (6) Compute the matrix $\mathbf{T} = \mathbf{U}_r \mathbf{\Sigma}_r^{-1/2}$.
- (7) Compute the matrix $\mathbf{\Phi} = \mathbf{X} \mathbf{T}$, which has modes $\hat{\phi}_i$ as columns,

$$\mathbf{\Phi} = \begin{bmatrix} | & | & & | \\ \hat{\phi}_1 & \hat{\phi}_2 & \dots & \hat{\phi}_r \\ | & | & & | \end{bmatrix} = \begin{bmatrix} | & | & & | \\ \hat{\mathbf{x}}_1 & \hat{\mathbf{x}}_2 & \dots & \hat{\mathbf{x}}_{m_{\mathbf{x}}} \\ | & | & & | \end{bmatrix} \mathbf{T}. \quad (6)$$

Unflatten all $\hat{\phi}_i \in \mathbb{R}^n$ to obtain modes $\phi_i \in V$.

The code to use modred to find the leading ten POD modes is shown below. The variable `weights` is a 1D or 2D numpy array and corresponds to the inner product weight matrix $\mathbf{W}$, `vecs` is a 2D numpy array and corresponds to $\mathbf{X}$ that is loaded from a text file. The variables `eig_vecs`, `eig_vals`, and `modes` correspond to $\mathbf{U}$, $\mathbf{\Sigma}$, and $\mathbf{\Phi}$, respectively, and all are numpy arrays.

```
vecs = numpy.loadtxt('vec_array.txt')
modes, sing_vals = modred.compute_POD_matrices_snap_method(vecs, range(10),
    inner_product_weights=weights)
```

In some cases, the data are small enough to take an SVD of $\mathbf{X}$ directly. This has the numerical advantage of not “squaring up” the eigenvalues as in the method of snapshots, which can cause numerical roundoff errors in modes which correspond to very small eigenvalues. In some cases, these modes are important, and so we provide another function which implements a slight variation of this method by directly taking the SVD of $\mathbf{X}$, `compute_POD_matrices_direct_method`. See the online documentation for more information on using this function.

Now we present the vector space approach, which eliminates the need to flatten the vectors and stack them as columns of $\mathbf{X}$. In fact, the matrix $\mathbf{X}$ is never formed.

- (1) Collect and store vectors $\mathbf{x}_i \in V$, for $i = 1, \dots, m_{\mathbf{x}}$.
- (2) Compute each entry of the $m_{\mathbf{x}} \times m_{\mathbf{x}}$ correlation matrix $\mathbf{H}$ via $[\mathbf{H}]_{i,j} = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$.
- (3) Compute the eigenvalues and eigenvectors of $\mathbf{H}$, writing $\mathbf{H}\mathbf{U} = \mathbf{U}\mathbf{\Sigma}$, where $\mathbf{\Sigma}$ is diagonal and real, and $\mathbf{U}$ is orthogonal (or unitary), since $\mathbf{H}$ is symmetric (or Hermitian). Sort the eigenvalues (and corresponding eigenvectors) in descending order.
- (4) Select the number of modes to keep, r . Truncate the matrices, keeping the first r columns of $\mathbf{U}$ to obtain $\mathbf{U}_r$, and the first r rows and columns of $\mathbf{\Sigma}$ to obtain $\mathbf{\Sigma}_r$.
- (5) Compute the matrix $\mathbf{T} = \mathbf{U}_r \mathbf{\Sigma}_r^{-1/2}$.
- (6) Construct modes $\boldsymbol{\varphi}_j$ individually via

$$\boldsymbol{\varphi}_j = \sum_{i=1}^{m_{\mathbf{x}}} \mathbf{x}_i [\mathbf{T}]_{i,j}, \quad j = 1, \dots, r. \quad (7)$$

The code to find the leading ten POD modes is shown here. The first argument to `PODHandles` is a function (callable) that takes two vectors and returns their inner product. The variables `vec_handles` and `mode_handles` are lists of so-called vector handles which are responsible for loading and saving vectors to disk. In the example, the vector handles work with files in Python's "pickle" binary format. In general, vector handles use very little memory and allow `modred` to load/save the large vectors so they are in memory only as they are needed. This is crucial for this size of data where not all vectors can be in memory simultaneously.

```
vec_handles = [PickleVecHandle('vec%d.fmt' % i) for i in range(100)]
mode_handles = [PickleVecHandle('mode%d.fmt' % i) for i in range(10)]
POD = modred.PODHandles(inner_product)
eig_vecs, eig_vals = POD.compute_decomp(vec_handles)
POD.compute_modes(range(10), mode_handles)
```

The most important difference between the two procedures is that the vector space approach involves no flattening and stacking of vectors. As a result, in step (3) of the matrix multiplication, the inner products involve the data matrix $\mathbf{X}$, but in step (2) of the vector space approach the inner products are computed one-by-one without data matrices. Similarly, step (7) of the matrix multiplication method results in a large matrix Φ of modes, but step (6) of the vector space method finds each mode independently via scalar multiplication and vector addition.

These two approaches each have advantages and disadvantages, and so `modred` includes implementations of both. As previously mentioned, for smaller and simpler datasets, the matrix multiplication approach and corresponding `PODArrays` class is both more computationally efficient and easier to use. For larger and more complicated datasets, the vector space approach and corresponding `PODHandles` class is easier to use and is parallelized. See Section 4.1 and Section 5 for an in-depth discussion of these topics.

2.2. BPOD

As mentioned in the introduction, Balanced POD is useful for finding reduced-order models of large linear input-output systems. A general form of a discrete-time linear system is

$$\begin{aligned} \mathbf{x}_{i+1} &= \mathbf{A}\mathbf{x}_i + \mathbf{B}\mathbf{u}_i \\ \mathbf{y}_i &= \mathbf{C}\mathbf{x}_i + \mathbf{D}\mathbf{u}_i, \end{aligned} \quad (8)$$

where $\mathbf{x}_i \in V$ is the state vector, $\mathbf{u}_i \in \mathcal{U}$ is regarded as an input, and $\mathbf{y}_i \in \mathcal{Y}$ as an output. Here, $\mathcal{U}$ and $\mathcal{Y}$ are vector spaces with respective dimensions p and q (i.e., p inputs and q outputs), and $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and $\mathbf{D}$ are linear operators between the appropriate spaces. As is often the case for large systems, BPOD is most useful when p and q are much less than n (the dimension of V). This represents a discrete-time formulation, but the equations are easily adapted to a continuous-time formulation, and the `modred` library handles both.

BPOD is an approximation of balanced truncation [Moore 1981], a popular model reduction procedure that has good a priori error bounds. In balanced truncation, one linearly transforms the state $\mathbf{x}_i$ such that the new coordinates are ranked in decreasing order of importance to the input-output dynamics. (More precisely, the states are ranked according to their degree of controllability and observability.) Truncating the resulting system by keeping only the first r states discards the states which are less important to the input-output dynamics.

Unfortunately, for systems with a large state dimension n it becomes computationally prohibitive to solve the Lyapunov equations associated with balanced truncation. BPOD approximates balanced truncation using the method of snapshots [Sirovich 1987], where the snapshots are sampled vectors from the linear system (8) and the corresponding adjoint system

$$\begin{aligned}\mathbf{z}_{i+1} &= \mathbf{A}^\dagger \mathbf{z}_i + \mathbf{C}^\dagger \mathbf{v}_i \\ \mathbf{w}_i &= \mathbf{B}^\dagger \mathbf{z}_i + \mathbf{D}^\dagger \mathbf{v}_i,\end{aligned}\tag{9}$$

where $\mathbf{z}_i \in V$, $\mathbf{v}_i \in \mathcal{Y}$, and $\mathbf{w}_i \in \mathcal{U}$. The adjoint operators, denoted $(\cdot)^\dagger$, depend on the inner products in the appropriate spaces: for instance, if $V = \mathbb{R}^n$ with an inner product $\langle \mathbf{z}, \mathbf{x} \rangle = \mathbf{z}^* \mathbf{W} \mathbf{x}$, where $\mathbf{W}$ is Hermitian positive definite, then

$$\mathbf{A}^\dagger = \mathbf{W}^{-1} \mathbf{A}^* \mathbf{W} \quad \mathbf{B}^\dagger = \mathbf{B}^* \mathbf{W} \quad \mathbf{C}^\dagger = \mathbf{W}^{-1} \mathbf{C}^*.\tag{10}$$

As inputs, BPOD takes snapshots sampled from impulse responses in each input, for both the linear system (8) and the adjoint system (9). The impulse response snapshots may be regarded equivalently as zero-input responses to initial conditions given for individual inputs and outputs. We collect a total of $m_{\mathbf{x}}$ “direct” (nonadjoint) snapshots and $m_{\mathbf{z}}$ adjoint snapshots, and thus $m_{\mathbf{x}}/p$ snapshots from each direct impulse response and $m_{\mathbf{z}}/q$ from each adjoint impulse response. The snapshots can be nonuniformly sampled if they are scaled by quadrature weights, but for simplicity, we assume the snapshots are uniformly sampled and omit the quadrature weights. BPOD produces two sets of modes that are ranked by their influence on the input-output dynamics (controllability and observability). The dynamics can then be projected onto the modes to find a reduced-order model that approximates the input-output dynamics (the dynamics in the most controllable and observable subspaces).

We describe the vector space approach for these computing BPOD modes. The matrix multiplication approach is analogous to POD. For more details, see Rowley [2005].

- (1) Collect and store snapshots $\mathbf{x}_j$, for $j = 1, \dots, m_{\mathbf{x}}$, and $\mathbf{z}_i$, for $i = 1, \dots, m_{\mathbf{z}}$.
- (2) Compute each entry of the Hankel matrix $\mathbf{H}$ via $[\mathbf{H}]_{i,j} = \langle \mathbf{z}_i, \mathbf{x}_j \rangle$.
- (3) Compute the singular value decomposition (SVD) $\mathbf{H} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^*$.
- (4) Select the number of modes to keep, r . Truncate the matrices, keeping the first r columns of $\mathbf{U}$ and $\mathbf{V}$ to obtain $\mathbf{U}_r$ and $\mathbf{V}_r$, and the first r rows and columns of $\mathbf{\Sigma}$ to obtain $\mathbf{\Sigma}_r$.
- (5) Compute the matrices $\mathbf{T}_{\mathbf{x}} = \mathbf{V}_r \mathbf{\Sigma}_r^{-1/2}$ and $\mathbf{T}_{\mathbf{z}} = \mathbf{U}_r \mathbf{\Sigma}_r^{-1/2}$.

- (6) Find the direct modes via $\varphi_j = \sum_{i=1}^{m_x} \mathbf{x}_i [\mathbf{T}_x]_{i,j}$ and the adjoint modes via $\psi_j = \sum_{i=1}^{m_z} \mathbf{z}_i [\mathbf{T}_z]_{i,j}$ for $j = 1, \dots, r$.

To clarify, this algorithm implements the balanced POD procedure given in Rowley [2005]. This method is sometimes confused with a related method given in Willcox and Peraire [2002], which is also based on snapshots of linearized and adjoint systems. In Willcox and Peraire [2002], however, the Hankel matrix in step (2) is not constructed; instead, one forms separate low-rank approximations of the controllability and observability Gramians (using two different SVDs) and then determines a transformation in which these approximate Gramians are equal and diagonal. This approach is inferior to the approach in Rowley [2005], particularly for systems with states that are weakly controllable but strongly observable or vice-versa. These are precisely the systems for which balancing provides the greatest benefit. For an example of the failure of the method in Willcox and Peraire [2002], see Figures 7 and 8 of Rowley [2005].

The code to perform BPOD with this approach is shown here. The primary difference from the POD case is there are now two sets of vector handles and mode handles, which correspond to $\mathbf{x}$, $\mathbf{z}$, φ , and ψ .

```
BPOD = BPODHandles(inner_product)
L_sing_vecs, sing_vals, R_sing_vecs = BPOD.compute_decomp(
    direct_vec_handles, adjoint_vec_handles)
BPOD.compute_direct_modes(range(10), direct_mode_handles)
BPOD.compute_adjoint_modes(range(10), adjoint_mode_handles)
```

2.3. DMD

Dynamic Mode Decomposition (DMD) is a useful for analyzing a series of data uniformly sampled in time, $\mathbf{x}_i = \mathbf{x}(t_i)$ for $i = 1, \dots, m_x$ and $t_i = i\Delta t$. The DMD modes φ_i and Ritz values λ_i are vectors and complex numbers, respectively, such that

$$\begin{aligned} \mathbf{x}_j &= \sum_{i=1}^{m_x-1} \lambda_i^{j-1} \varphi_i & j &= 1, \dots, m_x - 1 \\ \mathbf{x}_{m_x} &= \sum_{i=1}^{m_x-1} \lambda_i^{m_x-1} \varphi_i + r & r &\perp \text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_{m_x-1}\}. \end{aligned} \tag{11}$$

If the vectors $\{\mathbf{x}_i \mid i = 1, \dots, m_x\}$ are linearly independent, such φ_i and λ_i always exist, and are unique [Chen et al. 2012]. We observe that the above equation is the discrete-time equivalent of an eigenvector decomposition for the evolution of a linear system, with all multiplicative constants subsumed in the scaling of the modes φ_i . As such, the Ritz values λ_i tell us the growth rate and frequency associated with each mode. For linear systems, this information can be used to perform a stability analysis. For nonlinear systems, we can identify oscillatory structures on an attractor. See the works cited for examples [Rowley et al. 2009; Schmid 2010].

The algorithm for computing the DMD modes and Ritz values can be implemented in a way that is closely related to the POD algorithm [Schmid 2010]. Previous work described a new variant that is especially suitable for our library due to its low memory requirements [Tu and Rowley 2012]. Here, we improve upon this algorithm by further eliminating unnecessary inner products and linear combinations (see Appendix A for details). Our implementation of the vector space approach to the algorithm is summarized here. The matrix multiplication approach is very similar to that of POD (both via the method of snapshots and direct SVD), and so is omitted.

- (1) Collect and store snapshots $\mathbf{x}_i$, for $i = 1 \dots, m_{\mathbf{x}}$.
- (2) Compute each entry of the correlation matrix via $[\mathbf{H}]_{i,j} = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$, using all but the last snapshot (i and j have range $1, \dots, m_{\mathbf{x}} - 1$).
- (3) Compute the eigenvalues and eigenvectors of $\mathbf{H}$, writing $\mathbf{H}\mathbf{U} = \mathbf{U}\mathbf{\Sigma}$, where $\mathbf{\Sigma}$ is diagonal and real, and $\mathbf{U}$ is orthogonal (or unitary) since $\mathbf{H}$ is symmetric (or Hermitian). Sort the eigenvalues (and corresponding eigenvectors) in descending order.
- (4) Define sub-matrix $\mathbf{H}' = [\mathbf{H}]_{1:m_{\mathbf{x}}-1, 2:m_{\mathbf{x}}-1}$ (i.e., $\mathbf{H}$ with the first column removed), making use of the previously computed correlation matrix $\mathbf{H}$.
- (5) Form the column matrix $\mathbf{H}''$, via $[\mathbf{H}'']_j = \langle \mathbf{x}_{m_{\mathbf{x}}}, \mathbf{x}_j \rangle$, for $j = 1, \dots, m_{\mathbf{x}} - 1$.
- (6) Compute $\mathbf{M} = \mathbf{\Sigma}^{-1/2} \mathbf{U}^* [\mathbf{H}' \ \mathbf{H}''] \mathbf{U} \mathbf{\Sigma}^{-1/2}$. Solve the eigenvalue problem $\mathbf{M}\mathbf{V} = \mathbf{V}\mathbf{\Lambda}$, where the diagonal entries of $\mathbf{\Lambda}$ are the Ritz values λ .
- (7) Compute the matrix $\mathbf{T} = \mathbf{U}\mathbf{\Sigma}^{-1/2}\mathbf{V}\mathbf{D}$, where $\mathbf{D}$ is a diagonal matrix with diagonal $\mathbf{d} = (\mathbf{V}^*\mathbf{V})^{-1}\mathbf{V}^*\mathbf{\Sigma}^{-1/2}\mathbf{U}^*\mathbf{H}_{[1:m_{\mathbf{x}}-1,1]}$.
- (8) Construct modes individually via $\boldsymbol{\varphi}_j = \sum_{i=1}^{m_{\mathbf{x}}} \mathbf{x}_i [\mathbf{T}_{\mathbf{x}}]_{i,j}$.

The code to use modred to perform DMD in this fashion is shown here. The variables are analogous to the POD case, with the exception of `ritz_vals`, `mode_norms_squared`, and `build_coeffs`, which are all numpy arrays and correspond to λ_i , $\|\boldsymbol{\varphi}_i\|^2$, and $\mathbf{T}$, respectively.

```
DMD = modred.DMDHandles(inner_product)
ritz_vals, mode_norms_squared, build_coeffs = DMD.compute_decomp(vec_handles)
my_DMD.compute_modes(range(10), mode_handles)
```

3. REDUCED-ORDER MODELS AND SYSTEM IDENTIFICATION

3.1. Petrov-Galerkin Projection for Linear Systems

In the case that the snapshots are taken from a linear time invariant (LTI) system such as (8), a reduced-order model can be found by projecting the full dynamics onto the reduced set of modes via Petrov-Galerkin projection. Petrov-Galerkin projection may be used to obtain balanced POD models for nonlinear systems (see Section 3.4.1 of Rowley [2005] or Ilak et al. [2010]), although modred implements it only for linear systems. An in-depth discussion of the application of Petrov-Galerkin projection to model reduction problems is given in Bui-Thanh [2007]. We outline this technique now, first for the matrix approach.

First, we approximate the full state vector $\mathbf{x} \in V$ as a linear combination of the modes $\boldsymbol{\varphi}_j$, denoted as follows:

$$\mathbf{x} \approx \sum_{j=1}^r q_j \boldsymbol{\varphi}_j = \boldsymbol{\Phi} \mathbf{q}, \quad (12)$$

where $\mathbf{q} = (q_1, \dots, q_r) \in \mathbb{R}^r$ and $\boldsymbol{\Phi} : \mathbb{R}^r \rightarrow V$. $\boldsymbol{\Phi}$ is, again, a matrix whose columns are the modes $\hat{\boldsymbol{\varphi}}_i$ and $(\hat{\cdot})$ denotes the representation of the mode as a one-dimensional vector in $\mathbb{R}^n$.

A second set of modes $\hat{\boldsymbol{\psi}}_i$ is needed, and these form the columns of a matrix $\boldsymbol{\Psi}$. Left-multiplying by $\boldsymbol{\Psi}^\dagger$ maps V to $\mathbb{R}^r$, and is equivalent to the inner products

$$[\boldsymbol{\Psi}^\dagger \mathbf{x}]_j = \langle \boldsymbol{\psi}_j, \mathbf{x} \rangle, \quad j = 1, \dots, r. \quad (13)$$

In POD and DMD, the second set of modes is the same as the first set, $\boldsymbol{\Phi} = \boldsymbol{\Psi}$. In BPOD, the second set of modes is the set of adjoint modes. Inserting (12) into (8) and

multiplying by $\Psi^\dagger$ then gives the reduced-order model

$$\begin{aligned}\mathbf{q}_{i+1} &= \mathbf{A}_r \mathbf{q}_i + \mathbf{B}_r \mathbf{u}_i \\ \mathbf{y}_i &= \mathbf{C}_r \mathbf{q}_i + \mathbf{D} \mathbf{u}_i,\end{aligned}\tag{14}$$

where

$$\mathbf{A}_r \equiv (\Psi^\dagger \Phi)^{-1} \Psi^\dagger \mathbf{A} \Phi, \quad \mathbf{B}_r \equiv (\Psi^\dagger \Phi)^{-1} \Psi^\dagger \mathbf{B}, \quad \mathbf{C}_r \equiv \mathbf{C} \Phi, \tag{15}$$

and $\mathbf{A}_r : \mathbb{R}^r \rightarrow \mathbb{R}^r$ is an $r \times r$ matrix, $\mathbf{B}_r : \mathcal{U} \rightarrow \mathbb{R}^r$, and $\mathbf{C}_r : \mathbb{R}^r \rightarrow \mathcal{Y}$. For instance, if $\mathcal{U} = \mathbb{R}^p$, then $\mathbf{B}_r$ is an $r \times p$ matrix, and similarly for $\mathbf{C}_r$. The entries of $\mathbf{A}_r$ are computed by $[\mathbf{A}_r]_{i,j} = \langle \psi_i, \mathbf{A} \phi_j \rangle$. For large systems, one generally does not have access to the full $\mathbf{A}$ in matrix form, but the result of $\mathbf{A}$ operating on vector ϕ_j can be computed.

Also note that for POD and BPOD, the matrix $\Psi^\dagger \Phi$ is simply the identity matrix because the modes are orthonormal (for POD) or bi-orthogonal (for BPOD). In cases like these, a flag can be set in `modred` to avoid the unnecessary computation of this matrix and its inverse.

Here we summarize the steps to form the reduced-order matrices in (14) using the vector space approach, where the input space $\mathcal{U} = \mathbb{R}^p$ and the output space $\mathcal{Y} = \mathbb{R}^q$.

- (1) Advance each balancing mode, ϕ_i , one timestep, resulting in $\phi'_i = \mathbf{A} \phi_i$.
- (2) Compute matrix $\mathbf{A}_r$ by first computing $\mathbf{M}$ where $\mathbf{M}_{i,j} = \langle \psi_i, \phi'_j \rangle$. Then $\mathbf{A}_r = (\Psi^\dagger \Phi)^{-1} \mathbf{M}$.
- (3) Compute matrix $\mathbf{B}_r$ by first computing $\mathbf{M}$ where $\mathbf{M}_{i,j} = \langle \psi_i, \mathbf{b}_j \rangle$ and $\mathbf{b}_j$ denotes the j^{th} column of $\mathbf{B}$ (in the abstract setting, $\mathbf{b}_j = \mathbf{B} \mathbf{e}_j$, where $\mathbf{e}_j$ is a standard basis vector of $\mathbb{R}^p$). Then, $\mathbf{B}_r = (\Psi^\dagger \Phi)^{-1} \mathbf{M}$.
- (4) Compute matrix $\mathbf{C}_r$, where the j^{th} column is $\mathbf{C} \phi_j$.

Code to compute the Petrov-Galerkin projection onto a set of modes with the vector space approach is shown here. As in previous examples, `inner_product` is a function to take the inner product of two vectors. The variable `mode_handles[i]` corresponds to the mode ϕ_i , `A_on_modes_handles[i]` to $\mathbf{A} \phi_i$, `B_on_basis_handles[i]` to $\mathbf{b}_i$, and `C_on_modes[j]` to $\mathbf{C} \phi_j$. To do so with the matrix multiplication approach is very similar and omitted.

```
Galerkin_proj = modred.LTIGalerkinProjectionHandles(
    inner_product, mode_handles)
A_reduced, B_reduced, C_reduced = Galerkin_proj.compute_model(
    A_on_mode_handles, B_on_basis_handles, C_on_modes)
```

3.2. OKID

The Observer/Kalman Filter Identification (OKID) estimates the Markov parameters of any system based on arbitrary and noisy input-output signals [Juang et al. 1991], and has been used in a wide range of applications. The input-output dynamics of a linear system are completely characterized by its Markov parameters, and so these are useful for other system identification methods, including the Eigensystem Realization Algorithm discussed in the next section. For the system (8), where we now let $\mathcal{U} = \mathbb{R}^p$ and $\mathcal{Y} = \mathbb{R}^q$, the Markov parameters, $\{\mathbf{M}_i \mid i = 1, \dots, m\}$, are defined as

$$\mathbf{M}_0 = \mathbf{D}, \quad \mathbf{M}_i = \mathbf{C} \mathbf{A}^{i-1} \mathbf{B} \quad \text{for} \quad i = 1, 2, \dots, m. \tag{16}$$

The (r, c) th entry of $\mathbf{M}_i$ corresponds to the r th output and c th input and each $\mathbf{M}_i$ has dimensions $q \times p$. For large systems, the Markov parameters can be computed efficiently as impulse responses to each input individually.

Given arbitrary input-output signals, $\mathbf{u}_i$ and $\mathbf{y}_i$, for $i = 1, \dots, m$, a misguided approach would be to solve the matrix problem

$$[\mathbf{y}_0 \ \mathbf{y}_1 \ \dots \ \mathbf{y}_m] = [\mathbf{M}_0 \ \mathbf{M}_1 \ \dots \ \mathbf{M}_m] \underbrace{\begin{bmatrix} \mathbf{u}_0 & \mathbf{u}_1 & \dots & \mathbf{u}_m \\ & \mathbf{u}_0 & \dots & \mathbf{u}_{m-1} \\ & & \ddots & \vdots \\ & & & \mathbf{u}_0 \end{bmatrix}}_{\mathbf{U}}. \quad (17)$$

There are several reasons directly solving this equation is impractical. First, the matrices may be ill-conditioned. Second, the noise is not filtered and can significantly reduce accuracy. Last, it may require a long time-series of data to accurately capture the Markov parameters, increasing the computational cost.

OKID addresses all of these drawbacks. In short, the system is augmented with an asymptotically stable Kalman filter. One solves for the Markov parameters of this modified system, $\{\tilde{\mathbf{M}}_i \mid i = 1, \dots, m\}$, and then for the Markov parameters of the original system. The matrix equation which must be solved, rather than (17), is

$$[\mathbf{y}_0 \ \mathbf{y}_1 \ \dots \ \mathbf{y}_m] = [\tilde{\mathbf{M}}_0 \ \tilde{\mathbf{M}}_1 \ \dots \ \tilde{\mathbf{M}}_m] \mathbf{V}, \quad (18)$$

where $\mathbf{V}$ is defined later in (19). We reproduce the outline of the procedure here. For more details, see Juang [1994].

- (1) Collect inputs $\{\mathbf{u}_i \mid i = 1, \dots, m\}$ and outputs $\{\mathbf{y}_i \mid i = 1, \dots, m\}$ as in (17).
- (2) Select the number of Markov parameters to estimate, s .
- (3) Construct the $\mathbf{V}$ matrix as defined here

$$\mathbf{V} = \begin{bmatrix} \mathbf{u}_0 & \mathbf{u}_1 & \mathbf{u}_2 & \dots & \mathbf{u}_s & \dots & \mathbf{u}_m \\ & \mathbf{v}_0 & \mathbf{v}_1 & \dots & \mathbf{v}_{s-1} & \dots & \mathbf{v}_{m-1} \\ & & \mathbf{v}_0 & \dots & \mathbf{v}_{s-2} & \dots & \mathbf{v}_{m-2} \\ & & & \ddots & \vdots & & \vdots \\ & & & & \mathbf{v}_0 & \dots & \mathbf{v}_{m-s} \end{bmatrix}, \quad (19)$$

where $\mathbf{v}_i = \begin{bmatrix} \mathbf{u}_i \\ \mathbf{y}_i \end{bmatrix}$, column vectors with dimensions $(p + q) \times 1$. Matrix $\mathbf{V}$ has dimensions $(p + (p + q)s) \times (m + 1)$.

- (4) Solve, via least squares, for Markov parameters of observer system (transpose of (18))

$$\mathbf{V}^T [\tilde{\mathbf{M}}_0 \ \tilde{\mathbf{M}}_1 \ \dots \ \tilde{\mathbf{M}}_m]^T = [\mathbf{y}_0 \ \mathbf{y}_1 \ \dots \ \mathbf{y}_m]^T. \quad (20)$$

- (5) Solve for the Markov parameters of original system:

$$\mathbf{M}_0 = \mathbf{D} = \tilde{\mathbf{M}}_0, \quad \mathbf{M}_i = \tilde{\mathbf{M}}_i^{(1)} - \sum_{k=1}^i \tilde{\mathbf{M}}_k^{(2)} \mathbf{M}_{i-k} \text{ for } k = 1, 2, \dots, s$$

where for $i \geq 1$ the Markov parameters of the observer system are composed of two submatrices, $\tilde{\mathbf{M}}_i = [\tilde{\mathbf{M}}_i^{(1)} \ \tilde{\mathbf{M}}_i^{(2)}]$. Submatrix $\tilde{\mathbf{M}}_i^{(1)}$ has dimensions $q \times p$ and $\tilde{\mathbf{M}}_i^{(2)}$ has dimensions $q \times q$.

The code to estimate Markov parameters is shown here. The variable `Markovs` corresponds to $\{\mathbf{M}_i \mid i = 1, \dots, m\}$ and is a three-dimensional array with dimensions that correspond to the time-step, output, and input so that `Markovs[i]` is $\mathbf{M}_i$. Variables `inputs` and `outputs` are two-dimensional arrays such that `inputs[i]` is $\mathbf{u}_i$ and

outputs[i] is $\mathbf{y}_i$. The last argument to OKID is m , the number of Markov parameters to estimate.

```
Markovs = modred.OKID(inputs, outputs, 20)
```

3.3. ERA

The Eigensystem Realization Algorithm (ERA) is a method for finding reduced-order input-output models from a system's Markov parameters [Juang and Pappa 1985; Ho and Kalman 1965]. The resulting models are theoretically equivalent to those from BPOD, and therefore when a sufficiently long series of Markov parameters is taken the models are balanced [Ma et al. 2011]. The states are ordered by input-output importance and can be truncated just as in BPOD. However, ERA does not require adjoint vectors, full-state information, or the computation of modes, thereby reducing the computation time by orders of magnitude. This also makes ERA useful for finding linear input-output models from estimated Markov parameters (e.g., from OKID and experimental data) and from expensive, high-dimensional simulations. The linear models are amenable to control design.

The procedure to compute ERA models is summarized here. For more details, see Ma et al. [2011].

- (1) Collect $2N$ Markov parameters (N pairs) in the following series

$$\mathbf{M}_1, \mathbf{M}_2, \mathbf{M}_{P+1}, \mathbf{M}_{P+2}, \dots, \mathbf{M}_{(N-1)P+1}, \mathbf{M}_{(N-1)P+2}, \quad (21)$$

where $\mathbf{M}_i$ is defined in (16). The integer P is the number of time steps between pairs of Markov parameters.

- (2) Construct (generalized) Hankel matrices, $\mathbf{H}$ and $\mathbf{H}'$, where

$$\mathbf{H} = \begin{bmatrix} \mathbf{M}_1 & \mathbf{M}_{P+1} & \dots & \mathbf{M}_{m_c+1} \\ \mathbf{M}_{P+1} & \mathbf{M}_{2P+1} & \dots & \mathbf{M}_{(m_c+1)P+1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{M}_{m_o P+1} & \mathbf{M}_{(m_o+1)P+1} & \dots & \mathbf{M}_{(m_c+m_o)P+1} \end{bmatrix} \quad (22)$$

and

$$\mathbf{H}' = \begin{bmatrix} \mathbf{M}_2 & \mathbf{M}_{P+2} & \dots & \mathbf{M}_{m_c+2} \\ \mathbf{M}_{P+2} & \mathbf{M}_{2P+2} & \dots & \mathbf{M}_{(m_c+1)P+2} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{M}_{m_o P+2} & \mathbf{M}_{(m_o+1)P+2} & \dots & \mathbf{M}_{(m_c+m_o)P+2} \end{bmatrix}. \quad (23)$$

The parameters m_c and m_o determine how much relative weight is placed on controllability and observability, respectively. Often one uses all data and sets m_c and m_o to be equal, that is, $m_c = m_o = (N - 1)/2$.

- (3) Compute the SVD such that $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$.
- (4) Select order of the reduced-order model, r . Truncate the matrices, keeping the first r columns of $\mathbf{U}$ and $\mathbf{V}$ to obtain $\mathbf{U}_r$ and $\mathbf{V}_r$, and the first $r \times r$ block of $\mathbf{\Sigma}$ to obtain $\mathbf{\Sigma}_r$.
- (5) Compute reduced system matrices:

$$\begin{aligned} \mathbf{A}_r &= \mathbf{\Sigma}_r^{-1/2} \mathbf{U}_r^* \mathbf{H}' \mathbf{V}_r \mathbf{\Sigma}_r^{-1/2} \\ \mathbf{B}_r &= \text{the first } p \text{ columns of } \mathbf{\Sigma}_r^{1/2} \mathbf{V}_r^* \\ \mathbf{C}_r &= \text{the first } q \text{ rows of } \mathbf{U}_r \mathbf{\Sigma}_r^{1/2}. \end{aligned} \quad (24)$$

The reduced system is then the same as in (14).

Code to compute an ERA reduced-order model is shown. A_{reduced} , B_{reduced} , and C_{reduced} correspond to A_r , B_r , and C_r . The argument `Markovs` is the same format as in OKID, a three-dimensional numpy array such that `Markovs[i]` is M_i . The singular values, Σ , are stored in the variable `sing_vals`.

```
ERA = modred.ERA()
A_reduced, B_reduced, C_reduced = ERA.compute_model(Markovs, 50)
sing_vals = ERA.sing_vals
```

4. SOFTWARE DESIGN

4.1. Use with Data of Different Size and Complexity

In the design process, careful consideration has been given to users and problem types. We distinguish between three cases of problems, delineated by the size of the dataset. The first case has datasets which are smaller, meaning that they fit on a single node, and simpler, in terms of inner products and data format. While not always true, we observe that small data and simple data often coincide, and so we consider them one case. For example, such datasets might be generated from experimental measurements or small simulations on a personal computer. Here, the matrix multiplication approach has a few advantages. Since the data fits in memory, we stack it all into data matrices and use matrix multiplication for inner products, taking advantage of the efficiency of highly optimized libraries such as BLAS, LAPACK, and Intel MKL. The added complexity of defining an inner product weighting and flattening the data into columns of a matrix is often not difficult or computationally demanding for this type of data. The classes which implement this approach are not parallelized for distributed memory and are most efficient when the numpy backend makes use of multiple cores.

In the second case, the dataset is larger so that data matrices such as $\mathbf{X}$ and Φ are too large to fit in memory and thus a direct implementation of the steps (as in the POD Section 2.1) is not possible. Still, the individual vectors are small enough that a few can be in a single node's memory simultaneously. Often the inner product is more complicated because the data might be multidimensional, defined over a nonuniform grid, and/or generated by a large simulation performed on computer cluster. Similarly, the data may be saved in some more complicated format. For this case, the vector space approach is ideal because each step's computations require only two individual vectors in memory simultaneously. Additionally, it preserves the mathematical abstraction between vectors (represented as objects in the sense of object-oriented programming) and functions which operate on vectors. In this approach, `modred` operates on vector objects, independent of the underlying implementation, resulting in high-level, general, modular, testable, code that is easily applied to any data format. Further, since the inner products and linear combinations require only a few vectors in memory at once, we are able to parallelize the computations for distributed memory architectures and achieve excellent scaling (see the parallelization Section 5). These functions are often already implemented in an efficient way in existing software (e.g., in the original simulation that produced the data), and in this case the existing routines can be wrapped and used within Python and `modred`. It is possible to use this approach for smaller data as well, but we find it can be significantly slower than the matrix multiplication implementation.

The third and final case is datasets with vectors too large to fit in a single node's memory, and it is not yet available in `modred`. Developing such an implementation is possible and is considered in the design. See Section 5.4 for more details.

It is worth noting that these algorithms require the entire dataset to be available throughout the computations. This is not an additional requirement imposed by `modred`, but one that is inherent to the algorithms because they contain operations

that involve all of the vectors and fully-formed matrices. That is, these algorithms cannot be performed “on the fly”, i.e., while a simulation or experiment is producing the dataset.

4.2. Library-Wide Design Principles

We incorporate other elements of software design throughout `modred`. The first, and most important, is an object-oriented, high-level, easy-to-use, and flexible interface. This interface ensures that the user never needs to modify or be aware of anything deeper than the carefully chosen inputs and outputs of each function, as shown in the code samples in previous sections. Users can focus on the science of their particular application rather than implementation details. In addition to saving users’ time, the ease of use opens up model reduction to a larger audience who can use model reduction as a tool without understanding exactly how the algorithms work, just their inputs and outputs.

The library is carefully parallelized for distributed memory machines, not only for performance (as described in Section 5), but also for modularity and usability. After a user writes a script that uses `modred` in serial, in almost all cases the same script can be used in parallel with *no changes*. The user does not need to know how to write parallel code to run in parallel, making `modred` accessible to a wide audience. Parallel execution requires the freely available MPI Python module `mpi4py`, but the library works seamlessly in serial without it. Second, all of `modred`’s nontrivial parallelized functions (inner products and linear combinations) are isolated to a lower-level class used by all of the higher-level classes. This modularity greatly simplifies implementation and future extensions of `modred`.

The library has a modular organization of classes and functions. Every function’s inputs and outputs are carefully chosen to be intuitive and reflect its mathematical structure. The details of the implementation are irrelevant to other classes and functions, and therefore improvements which change the implementation, for example to increase efficiency, have little, if any, effect on other code that uses `modred`, including users’ scripts. This organization also enables `modred` to have comprehensive unit tests for each class method and function. This is a major advantage over “home grown” numerical packages that may not have such comprehensive automated testing. Instead, by testing all functions and steps of algorithms independently for a variety of inputs and expected outputs, the results are much more likely to be correct. This type of testing is widely employed for large projects, and it greatly improves reliability and eases debugging by isolating potential problems. In addition, we unit test all parallelized functions in serial and in parallel.

The computational overhead of `modred` is quite low since its primary purpose is to call `numpy` in the matrix multiplication case and orchestrate calls of the user-supplied functions in the vector space case. Simply put, if `numpy` and the user-supplied functions are fast, then so is `modred`. A common concern is that user-supplied functions, written in Python, are not executed fast in comparison to compiled languages such as C/C++ and Fortran. With appropriate usage of Python and `numpy` (e.g., avoiding loops), the speed limitations of Python are often acceptable. For cases where the speed limitations are severe, there are several packages which make it easy to mix compiled languages with Python, including `Cython` and `SWIG` for C/C++ and `f2py` for Fortran. Using these, one can write the inner product, loading, and saving functions in a compiled language, wrap them into a Python module, and use them with `modred`. In cases where these functions already exist, for example as part of simulation software, users can wrap the existing functions into a Python module and save themselves from tedious and mistake-prone duplication and/or translation.

5. PARALLELIZATION

In this section, we describe how we parallelize the vector operations used in the vector space approach and corresponding implementations of POD, BPOD, DMD, and Petrov-Galerkin projections for a distributed memory architecture via MPI. There are two main steps in each decomposition: computing inner products and taking linear combinations of the input vectors to form the modes. The matrix multiplication and array-based implementations are not parallelized and operate best when numpy is installed with a shared memory backend such as Intel's MKL. Similarly, the system identification methods OKID and ERA are generally not computationally intensive and thus are not parallelized.

5.1. Inner Product Matrices

The computation of inner products of vectors is often the most expensive step of modal decompositions. However, it is easily parallelized since each element of $\mathbf{H}$ depends only on the inner product of two vectors, $[\mathbf{H}]_{i,j} = \langle \mathbf{z}_i, \mathbf{x}_j \rangle$. Throughout this section, the rows of $\mathbf{H}$ are indexed by i and there are n_r of them. Similarly, the columns are indexed by j and there are n_c of them.

Now we outline the parallelization of computing the inner product matrix. First, the rows and columns of $\mathbf{H}$ (and corresponding vectors $\mathbf{z}_i$ and $\mathbf{x}_j$) are assigned to the n_w workers. Thus, the first n_r/n_w rows and n_c/n_w columns are assigned to the first worker, the second n_r/n_w rows and n_c/n_w columns are assigned to the second worker, and so on.

Often there are too many vectors to fit in memory simultaneously, so we load only subsets of at one time. We define the maximum number of vectors which can be in memory as n_v . Thus, each subset consists of $n_v - 2$ vectors, where the 2 comes from also loading a vector corresponding to a column and to an extra spot as a buffer for MPI send and receives (upcoming). In the first, outermost, loop, each worker iterates over subsets of modes, of which there are $n_r/(n_w(n_v - 2))$. The workers load the vectors that correspond to their current subset of rows, $\mathbf{z}_i$.

In the first nested loop, each worker loads a single vector corresponding to a column, $\mathbf{x}_j$. At this point, $n_v - 1$ vectors are in each worker's local memory. In the second nested loop, the inner products are computed. On the first iteration, the inner products are computed between the vectors corresponding to the loaded rows, $\mathbf{z}_i$, and the newly loaded vector $\mathbf{x}_j$. On subsequent iterations, all of the workers exchange $\mathbf{x}_j$ via MPI sends and receives in a circular pattern. During the MPI send and receives, there are at most n_v vectors in each worker's local memory. Then workers compute the inner products of the loaded vectors $\mathbf{z}_i$ (unchanged) and the newly received vector $\mathbf{x}_j$, filling in new parts of a column in matrix $\mathbf{H}$. This process is repeated until the circular pattern of MPI sends and receives is completed.

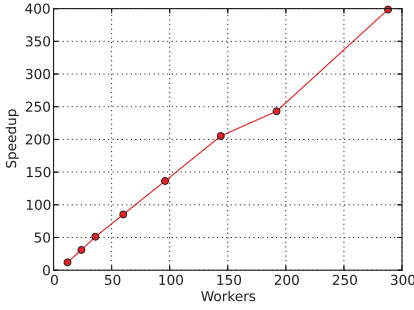
After the completion of these three loops, each worker has a portion of the $\mathbf{H}$ matrix filled in. Finally, an MPI allreduce sums each worker's $\mathbf{H}$ with all others', resulting in the completed $\mathbf{H}$ matrix in each worker's local memory.

This procedure's scaling is shown in Table I. Overall, it scales linearly with n_w . The number of loads has a term which scales quadratically with n_w and corresponds to n_c . This quadratic term exists because increasing n_w decreases the number of total loads. To exploit the quadratic scaling, we always use $n_c > n_r$ by transposing the rows and columns when necessary. We also note that when there is a fixed amount of memory (i.e., $n_v n_w$ is constant), as is the case for multiple processors on a node, it is typically advantageous to use all available workers (also see the Section 5.3 on hybrid parallelization).

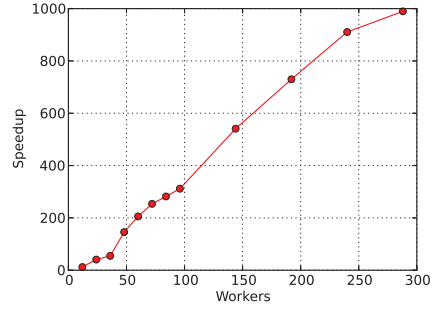
Benchmark results are summarized in Figure 1(a). The vectors are composed of 9,000 random double-precision floating-point numbers and are saved to file in Python's binary

Table I. Scaling of Inner Products

Operation	No. operations per worker	Wall time scaling
Loads	$\left(\frac{n_r}{(n_v - 2)n_w}\right) \left(\frac{n_c}{n_w}\right) + \frac{n_r}{n_w}$	$O\left(\frac{n_c n_r}{n_v n_w^2}\right) + O\left(\frac{n_r}{n_w}\right)$
Send-receives	$(n_w - 1) \left(\frac{n_c}{n_w}\right) \left(\frac{n_r}{(n_v - 2)n_w}\right)$	$O\left(\frac{n_r n_c}{n_v n_w}\right)$
Inner products	$\frac{n_r n_c}{n_w}$	$O\left(\frac{n_r n_c}{n_w}\right)$



(a) Inner products.



(b) Linear combinations.

Fig. 1. Measured scaling of parallelized vector operations.

“pickle” format. The number of rows and columns is $n_r = n_c = 8000$, and the maximum number of vectors in each worker’s local memory is $n_v = 5$. Typically, n_v would be larger for this size vector, but we use this lower value just for demonstration. The figure shows that increasing the number of workers results in a roughly linear speedup. The slope of this speedup is greater than one, meaning increasing the number of workers is highly advantageous. This is due to the quadratic scaling in the number of loads.

When $\mathbf{H}$ is symmetric, as in POD and DMD, the number of inner products to compute is nearly halved. We use a more complicated parallelization to take advantage of this, but the scaling is the same and so we omit the details.

5.2. Linear Combinations

The second parallelized portion of the code is the computation of the modes from linear combinations of the snapshots, following (1). In this section, we let n_s be the number of snapshots and n_m be the number of modes.

First, the snapshots and modes (not yet computed) are assigned to the n_w workers. Thus, the first n_s/n_w snapshots and n_m/n_w modes are assigned to the first worker, the second n_s/n_w snapshots and n_m/n_w modes are assigned to the second worker, and so on. In the outermost, first, loop, each worker computes $n_v - 2$ modes. In the first nested loop, each worker loads one snapshot $\mathbf{x}_j$ from the set it is responsible for (so n_s/n_w iterations). In the second nested loop, the snapshots are summed to form the modes $\boldsymbol{\varphi}_j$. On the first iteration, the newly loaded snapshot’s contribution ($\mathbf{x}_j[\mathbf{T}]_{i,j}$) is summed to the subset of modes currently in memory. It can help to think of these contributions to the modes as “layers”, and the sum of all layers is the mode. On subsequent iterations, all of the workers exchange their snapshots via MPI sends and receives in a circular pattern. During the send and receives, there are at most n_v vectors in each worker’s local memory. After exchanging snapshots, each worker computes a new subset of mode

Table II. Scaling of Operations for Computing Linear Combinations

Operation	No. operations per worker	Wall time scaling
Loads	$\left(\frac{n_s}{n_w}\right) \left(\frac{n_m}{(n_v - 2)n_w}\right)$	$O\left(\frac{n_s n_m}{n_v n_w^2}\right)$
Send-receives	$(n_w - 1) \left(\frac{n_s}{n_w}\right) \left(\frac{n_m}{(n_v - 2)n_w}\right)$	$O\left(\frac{n_s n_m}{n_v n_w}\right)$
Scalar multiplications	$\frac{n_s n_m}{n_w}$	$O\left(\frac{n_s n_m}{n_w}\right)$

layers that correspond to the newly received snapshot. This process is repeated until the circular pattern of MPI sends and receives is completed.

At the completion of first nested loop, each worker has a completed subset of modes and saves them to disk. Then a new subset of modes is begun (another iteration of the outermost loop), until all modes are computed and saved.

This procedure's scaling is shown in Table II. The overall scaling is very similar to the scaling of the inner products. First, the scaling is linear with n_w . Again, loading (usually the most time-consuming operation) scales quadratically because additional workers reduce the total number of snapshot loads. Just as for inner products, when there is a fixed amount of memory, that is, $n_v n_w$ is constant as is the case for multiple processors on a node, it is typically advantageous to use all available workers (also see the Section 5.3 on hybrid parallelization).

The scaling results for a dataset similar to the one used for the inner product computations are shown in Figure 1(b). The figure shows that increasing the number of workers results in a roughly linear speedup. The slope of this speedup is greater than one, and as for the inner product case, this is due to the quadratic scaling of the loading.

5.3. Hybrid Parallelization

The modred library itself is only parallelized for distributed memory (via MPI). However, it is possible to achieve speedups by using shared memory within a node. Each MPI worker can be an entire node. Then, the user-supplied functions for inner products, loading, and saving can be written for a shared memory architecture (e.g., via OpenMP) and use all of the processors on each node. This approach could improve efficiency, but would depend on the type of data, the operations involved, and the particular computer hardware being used.

5.4. Very Large Data Parallelization

Currently modred cannot operate on vectors so large that three cannot fit in a node's memory simultaneously. A work-around is to use so-called "fat" nodes – nodes with large amounts of memory, thereby increasing n_v to at least three. The algorithms' speeds are limited more by memory than processing speed, so fat nodes are generally a good choice. In the future, modred could be extended to operate on this data, for example, by generalizing MPI workers to be MPI communicators and allowing the user-provided vector class and inner product function to themselves operate in a distributed way across multiple nodes. The modular design makes this extension simpler.

6. EXAMPLE RESULTS

In this section, we show two example usages of modred on real problems of interest and all the intricacies associated with such real problems. The modred library performs well, reproducing existing model-reduction results that accurately describe the governing systems.

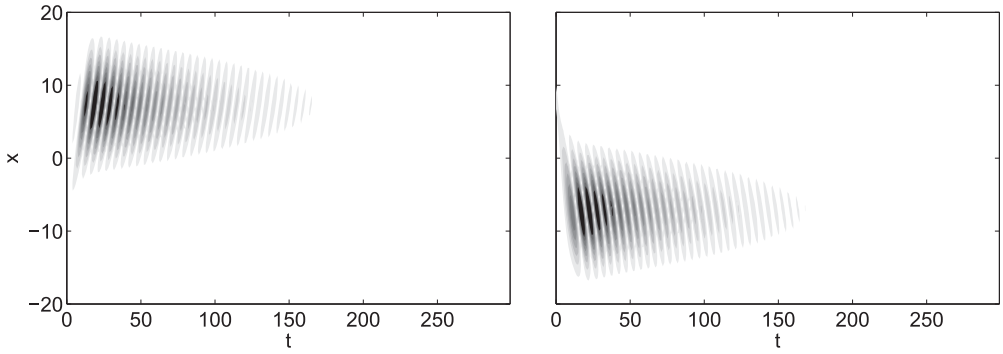


Fig. 2. Left: Real part of snapshots of impulse response of the direct system to the optimal disturbance input. Right: Real part of snapshots of impulse response of the adjoint system to the optimal disturbance input.

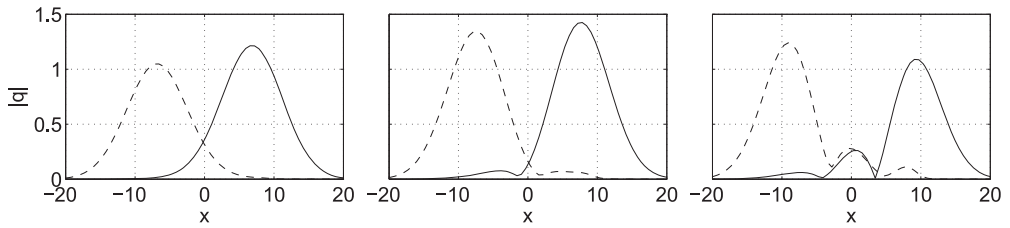


Fig. 3. Absolute value of the BPOD direct (solid lines) and adjoint (dashed lines) modes with the optimal disturbance and a sensor centered about $x = 8.24$.

6.1. Smaller Data: Complex Ginzburg-Landau Equation

In this case, the system is the linearized complex Ginzburg-Landau (CGL) equation, a partial differential equation in time, t , and one spatial dimension, x . This system displays many physical phenomena, including convection and diffusion, and thus is frequently used as a model for instabilities in fluid systems [Huerre and Monkewitz 1990]. We demonstrate that our library is effective by reproducing the calculation of BPOD modes and reduced-order model of the subcritical case in previous work [Ilak et al. 2010]. We use the same numerical approach as in previous work, based on spectral differentiation and Hermite polynomials, translated from the MATLAB library described in Weideman and Reddy [2000] to Python. All of the code is provided in the library's examples for convenience. Figure 2 shows the time evolution of an impulse response to the direct and adjoint systems.

The system is relatively small, with the complex solution discretized over 220 points. Thus the matrix multiplication and array implementations are appropriate. The grid is non-uniform and so there is a weighted inner product. The resulting BPOD modes are shown in Figure 3, and the impulse response of the reduced-order model is shown to be indistinguishable from that of the full system in Figure 4. The same result is shown in the original work, and the results obtained with *modred* are consistent with these.

6.2. Larger Data: Boundary Layer

The physical system is a spatially evolving boundary layer over a flat plate in two spatial dimensions (x and y) and is governed by the linearized incompressible Navier-Stokes partial differential equations. The application is to use feedback control to dampen the natural growth of disturbances. To do so, the controller is designed to

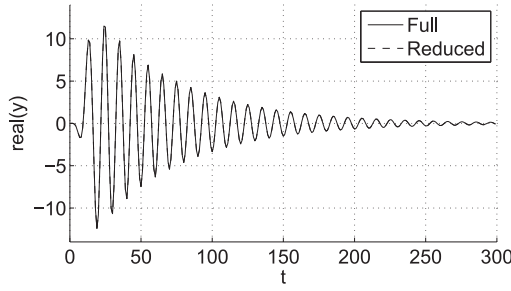


Fig. 4. The real component of the impulse responses of full and reduced systems. The maximum difference between the two is $2.0 \cdot 10^{-6}$.

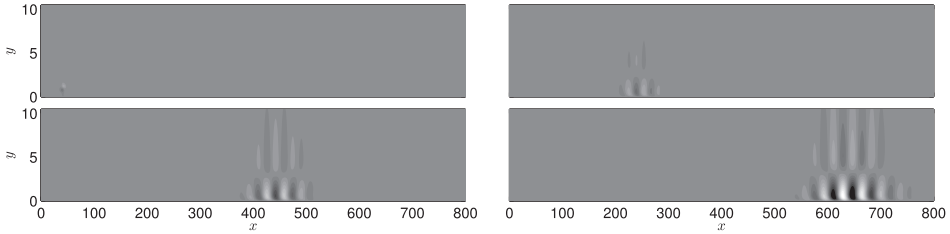


Fig. 5. Snapshots of the response to an impulse in the disturbance input sampled at intervals of 500 convective time units.

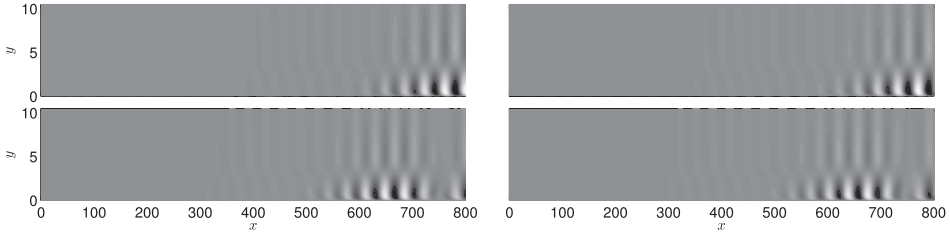


Fig. 6. Leading four POD modes.

minimize the energy in the flow, as approximated by the projection of the state onto the most energetic (POD) modes [Belson et al. 2013a]. The flow is directly numerically simulated with a pseudo-spectral solver [Chevalier et al. 2007]. Figure 5 shows the snapshots from an impulse response in the inputs (located along the horizontal axis at 35 and 400). Approximately 4000 snapshots are used, each of which contains the two components of velocity at 768×101 grid points and occupies roughly two megabytes of memory.

In this case, the size of the data and computer's memory make it impossible to fit all of the vectors in memory simultaneously. Furthermore, the inner product is a complicated function involving discrete Fourier transforms and is more easily represented as a function of two vectors than as a weighted matrix. Thus, the POD modes are computed using the vector space approach and the `PODHandles` class and custom vector and vector handle classes (including the inner product). The original simulation software is written in Fortran, so the `f2py` package is used to wrap existing subroutines to make them accessible to Python [Peterson 2009]. Figure 6 shows the resulting POD modes.

Without `modred`, performing POD on this data would likely be accomplished by flattening and stacking all of the vectors into matrices, and performing a large matrix multiplication. However, this could require writing the inner product weights explicitly rather than as a function, which is laborious. It also would require a computer with

more memory—enough to have all of the snapshots in memory simultaneously. Furthermore, the automated testing provided with `modred` helps ensure the accuracy of the results; without such comprehensive testing, subtle programming errors can often go undetected if the results look reasonable.

7. CONCLUSIONS

As datasets have grown larger, so has the need for approximating them with low-dimensional models. We present a new Python library for model reduction, modal decompositions, and system identification. A few modal decomposition methods, along with Petrov-Galerkin projection of linear systems onto modes, are implemented. This library fills a need for a publicly available implementation of these algorithms.

Two implementations of these algorithms are provided for different categories of data. One implementation follows the common matrix multiplication approach and is suited for smaller and simpler data. The vectors are flattened and stacked into matrices to make use of highly optimized matrix multiplication algorithms. A second implementation follows a vector space approach and is well suited for larger and more complicated data. This approach can handle any data format since the user provides a vector handle for loading and saving vector objects. Since Python is dynamically typed, the only requirements of the vector objects are to satisfy the mathematical properties of a vector, namely addition, scalar multiplication, and inner products. These vectors are loaded and saved only as needed within `modred` by user-supplied vector handles, which themselves use very little memory. As a result, `modred` can handle large datasets and is parallelized for a distributed memory architecture with favorable scaling.

The library itself is lightweight, and the computational speed is limited only by the speed of the SVDs, eigenvalue decompositions, and user-supplied functions. Additionally, it is easy for users to wrap compiled code (e.g., C/C++ and Fortran) into Python modules for increased speed. Thus the speed limitations of Python can be bypassed as necessary.

The system identification methods ERA and OKID are also implemented. These are not computationally intensive and are not parallelized. To our knowledge, this is the first widely available implementation of these widely applicable methods.

The library is written in a modular way to make it easier to use, test, and extend in the future. All of the classes and member functions are carefully designed to mirror the mathematical algorithms, promoting clarity and understanding. Each function is tested independently for a range of cases, checking sources of programming errors. Thus, when the tests pass, one can have confidence in the results. This contrasts with in-house codes that may not be thoroughly tested at the level of individual functions.

The modular, object-oriented, design makes it easy for a user to use any or all of the aspects of `modred`. A user does not need to know all of the details of the underlying algorithm and implementation, only a few function calls, and we hope this will make model reduction a tool accessible to a broader audience.

In the future, we anticipate others will contribute additional useful model reduction techniques and collaborate with the development of the library. It is available under the Free BSD license, and is publicly available [Belson et al. 2013b].

APPENDIX

A. DMD ALGORITHM DERIVATION

There are a number of algorithms for computing DMD modes, each an improvement on its predecessor. While the companion-matrix formulation provides a clear analogy to Koopman spectral analysis [Rowley et al. 2009], the SVD-based algorithm given

in Schmid [2010] is more computationally stable. A low-memory variant of the latter algorithm requires as few as two vectors in memory at any given time, making it suitable for large data [Tu and Rowley 2012]. It also eliminates a large number of unnecessary inner products by reusing elements of the correlation matrix in later computations (about a 50% reduction). Here, we improve on the low-memory algorithm by identifying and eliminating additional redundancies, avoiding costly inner products and linear combinations of snapshots. Specifically, whenever possible we replace operations involving snapshots with equivalent operations using small matrices. For large data, this is much more efficient, as the small matrices can be stored in memory and manipulated easily, whereas the snapshots require careful memory management.

The original low-memory DMD algorithm is summarized here.

- (1) Collect and store snapshots $\mathbf{x}_i$, for $i = 1, \dots, m_{\mathbf{x}}$.
- (2) Compute each entry of the correlation matrix via $[\mathbf{H}]_{i,j} = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$, using all but the last snapshot (i and j have range $1, \dots, m_{\mathbf{x}} - 1$).
- (3) Compute the eigenvalues and eigenvectors of $\mathbf{H}$, writing $\mathbf{H}\mathbf{U} = \mathbf{U}\mathbf{\Sigma}$, where $\mathbf{\Sigma}$ is diagonal and real, and $\mathbf{U}$ is orthogonal (or unitary) since $\mathbf{H}$ is symmetric (or Hermitian). Sort the eigenvalues (and corresponding eigenvectors) in descending order.
- (4) Construct the POD modes individually via $\boldsymbol{\psi}_j = \sum_{i=1}^{m_{\mathbf{x}}} \mathbf{x}_i [\mathbf{U}\mathbf{\Sigma}^{-1/2}]_{i,j}$.
- (5) Define the submatrix $\mathbf{H}' = [\mathbf{H}]_{1:m_{\mathbf{x}}-1, 2:m_{\mathbf{x}}-1}$ (i.e., $\mathbf{H}$ with the first column removed), making use of the previously computed correlation matrix.
- (6) Compute each entry of $\mathbf{H}''$ via $[\mathbf{H}'']_i = \langle \mathbf{x}_i, \mathbf{x}_{m_{\mathbf{x}}} \rangle$.
- (7) Compute $\mathbf{M} = \mathbf{\Sigma}^{-1/2} \mathbf{U}^* [\mathbf{H}' \ \mathbf{H}''] \mathbf{U} \mathbf{\Sigma}^{-1/2}$ and solve the eigenvalue problem $\mathbf{M}\mathbf{V} = \mathbf{\Lambda}\mathbf{V}$.
- (8) Construct intermediate (unscaled) modes individually via $\tilde{\boldsymbol{\phi}}_j = \sum_{i=1}^{m_{\mathbf{x}}} \boldsymbol{\psi}_i [\mathbf{V}]_{i,j}$.
- (9) Compute the elements of $\mathbf{d}'$ via $[\mathbf{d}']_i = \langle \tilde{\boldsymbol{\phi}}_i, \mathbf{x}_1 \rangle$.
- (10) Compute the vector $\mathbf{d} = (\mathbf{V}^* \mathbf{V})^{-1} \mathbf{d}'$.
- (11) Scale the DMD modes: $\boldsymbol{\phi}_i = [\mathbf{d}]_i \tilde{\boldsymbol{\phi}}_i$.

(Note that we have changed from the notation used in Tu and Rowley [2012] to better match that used in this work.)

In this algorithm, there are five steps (2, 4, 8, 9, 11) that require manipulations of snapshots (or equivalently large data). Many of these computations can be combined or avoided completely. To see this, we first note that the “intermediate” modes in Step (9) can be written in terms of the original vectors as

$$\begin{aligned}
 \tilde{\boldsymbol{\phi}}_j &= \sum_{i=1}^{m_{\mathbf{x}}} \boldsymbol{\psi}_i [\mathbf{V}]_{i,j} \\
 &= \sum_{i=1}^{m_{\mathbf{x}}} \sum_{k=1}^{m_{\mathbf{x}}} \mathbf{x}_k [\mathbf{U}\mathbf{\Sigma}^{-1/2}]_{k,i} [\mathbf{V}]_{i,j} \\
 &= \sum_{k=1}^{m_{\mathbf{x}}} \mathbf{x}_k \sum_{i=1}^{m_{\mathbf{x}}} [\mathbf{U}\mathbf{\Sigma}^{-1/2}]_{k,i} [\mathbf{V}]_{i,j} \\
 &= \sum_{k=1}^{m_{\mathbf{x}}} \mathbf{x}_k [\mathbf{U}\mathbf{\Sigma}^{-1/2} \mathbf{V}]_{k,j}.
 \end{aligned} \tag{25}$$

We achieve a savings by moving $\mathbf{x}_k$ outside of one of the sums; to compute this linear combination, the snapshots only have to be loaded and summed once. Previously, the algorithm required two independent linear combinations (Steps (4) and (8)).

We achieve further savings by eliminating unnecessary inner products from the computation of $\mathbf{d}'$. Substituting for $\tilde{\varphi}_i$, we find

$$\begin{aligned} [\mathbf{d}']_i &= \langle \tilde{\varphi}_i, \mathbf{x}_1 \rangle \\ &= \left\langle \sum_{j=1}^{m_{\mathbf{x}}} \mathbf{x}_j [\mathbf{U}\Sigma^{-1/2}\mathbf{V}]_{j,i}, \mathbf{x}_1 \right\rangle \\ &= \sum_{j=1}^{m_{\mathbf{x}}} [\mathbf{U}\Sigma^{-1/2}\mathbf{V}]_{j,i} \langle \mathbf{x}_j, \mathbf{x}_1 \rangle. \end{aligned}$$

The inner products $\langle \mathbf{x}_j, \mathbf{x}_1 \rangle$ are simply elements of $\mathbf{H}$, which has already been computed in Step (2). This eliminates the m inner products required in Step (9). As such, we can write

$$\mathbf{d}' = \mathbf{V}^* \Sigma^{-1/2} \mathbf{U}^* [\mathbf{H}]_{1:m_{\mathbf{x}}-1,1}, \quad (26)$$

which is a product of small matrices. We can then compute $\mathbf{d}$ as

$$\begin{aligned} \mathbf{d} &= (\mathbf{V}^* \mathbf{V})^{-1} \mathbf{d}' \\ &= (\mathbf{V}^* \mathbf{V})^{-1} \mathbf{V}^* \Sigma^{-1/2} \mathbf{U}^* [\mathbf{H}]_{1:m_{\mathbf{x}}-1,1}, \end{aligned} \quad (27)$$

again using only previously computed matrices.

Finally, combining the above results, we can write

$$\varphi_j = \sum_{i=1}^{m_{\mathbf{x}}} \mathbf{x}_i [\mathbf{T}_{\mathbf{x}}]_{i,j}, \quad (28)$$

where

$$\mathbf{T}_{\mathbf{x}} = \mathbf{U} \Sigma^{-1/2} \mathbf{V} \mathbf{D}, \quad (29)$$

and $\mathbf{D} = \text{diag}(\mathbf{d})$, a diagonal matrix with main diagonal $\mathbf{d}$. Thus, we can compute the scaled DMD modes using a single linear combination operation; the original low-memory algorithm required computing intermediate modes, computing a scaling, and then scaling the modes. In summary, this improved formulation reduces the total number of linear combinations by half and the total number of inner products by m . (We note that because the algorithm requires $O(m^2)$ total inner products, the latter is a relatively minor savings.)

REFERENCES

- S. Ahuja and C. W. Rowley. 2010. Feedback control of unstable steady states of flow past a flat plate using reduced-order estimators. *J. Fluid Mech.* 645, 447–478.
- N. Aubry, P. Holmes, J. L. Lumley, and E. Stone. 1988. The dynamics of coherent structures in the wall region of a turbulent boundary layer. *J. Fluid Mech.* 192, 115–173.
- M. F. A. Azeez and A. F. Vakakis. 2000. Proper orthogonal decomposition (POD) of a class of vibroimpact oscillations. *J. Sound Vib.* 240, 5, 859–889.
- B. A. Belson, O. Semeraro, C. W. Rowley, and D. S. Henningson. 2013a. Feedback control of instabilities in the 2D Blasius boundary layer: The role of sensors and actuators. *Phys. Fluids* 25, 5, 054106.
- B. A. Belson, J. H. Tu, and C. W. Rowley. 2013b. Modred website. <http://pypi.python.org/pypi/modred> (Last accessed 11/13).
- S. L. Brunton, C. W. Rowley, and D. R. Williams. 2013. Reduced-order unsteady aerodynamic models at low Reynolds numbers. *J. Fluid Mech.* 724, 203–233.
- T. Bui-Thanh. 2007. Model-constrained optimization methods for reduction of parameterized large-scale systems. Ph.D. Dissertation, Massachusetts Institute of Technology.
- K. K. Chen, J. H. Tu, and C. W. Rowley. 2012. Variants of dynamic mode decomposition: Boundary condition, Koopman, and Fourier analyses. *J. Nonlinear Sci.* 22, 6, 887–915.

- M. Chevalier, P. Schlatter, A. Lundbladh, and D. S. Henningson. 2007. SIMSON: A pseudo-spectral solver for incompressible boundary layer flows. Tech. Rep., KTH.
- G. Dergham, D. Sipp, and J.-C. Robinet. 2011. Accurate low dimensional models for deterministic fluid systems driven by uncertain forcing. *Phys. Fluids* 23, 094101.
- B. L. Ho and R. E. Kalman. 1965. Effective construction of linear state-variable models from input/output data. In *Proceedings of the 3rd Annual Allerton Conference on Circuit and System Theory*.
- P. J. Holmes, J. L. Lumley, G. Berkooz, and C. W. Rowley. 2011. *Turbulence, Coherent Structures, Dynamical Systems and Symmetry* 2nd Ed. Cambridge University Press, Cambridge, UK.
- J. R. Holton. 2004. *An Introduction to Dynamic Meteorology*. Volume 1, 4th Ed. Academic Press Inc, Burlington, MA.
- K. H. Hsieh, M. W. Halling, and P. J. Barr. 2006. Overview of vibrational structural health monitoring with representative case studies. *J. Bridge. Eng.* 11, 6, 707–715.
- P. Huerre and P. A. Monkewitz. 1990. Local and global instabilities in spatially developing flows. *Annu. Rev. Fluid Mech.* 22, 473–537. DOI: <http://dx.doi.org/10.1146/annurev.fluid.22.1.473>
- M. Ilak, S. Bagheri, L. Brandt, C. W. Rowley, and D. S. Henningson. 2010. Model reduction of the nonlinear complex Ginzburg-Landau equation. *SIAM J. Appl. Dyn. Sys.* 9, 4, 1284–1302. DOI: <http://dx.doi.org/10.1137/100787350>
- M. Ilak and C. W. Rowley. 2008. Modeling of transitional channel flow using balanced proper orthogonal decomposition. *Phys. Fluids* 20, 034103.
- J.-N. Juang. 1994. *Applied System Identification*. Prentice-Hall, Upper Saddle River, NJ.
- J.-N. Juang and R. S. Pappa. 1985. An eigensystem realization algorithm for modal parameter identification and model reduction. *J. Guid. Control Dynam.* 8, 5, 620–627.
- J.-N. Juang, M. Phan, L. G. Horta, and R. W. Longman. 1991. Identification of observer/Kalman filter Markov parameters: Theory and experiments. NASA Tech. Mem. 104069.
- Y. C. Liang, W. Z. Lin, H. P. Lee, S. P. Lim, K. H. Lee, and H. Sun. 2002. Proper orthogonal decomposition and its applications – Part 2: Model reduction for MEMS dynamical analysis. *J. Sound Vib.* 256, 3, 515–532.
- M. Loève. 1955. *Probability Theory*. Springer-Verlag, New York, NY.
- J. L. Lumley. 1967. The structure of inhomogeneous turbulence. In *Atmospheric Turbulence and Wave Propagation*, A. M. Yaglom and V. I. Tatarski, Eds., Nauka, Moscow, 166–178.
- Z. Ma, S. Ahuja, and C. W. Rowley. 2011. Reduced order models for control of fluids using the Eigensystem Realization Algorithm. *Theor. Comput. Fluid Dyn.* 25, 1, 233–247.
- B. C. Moore. 1981. Principal component analysis in linear systems: Controllability, observability, and model reduction. *IEEE Trans. Automat. Contr.* 26, 1, 17–32.
- K. Pearson. 1901. On lines and planes of closest fit to systems of points in space. *Philos. Mag.* 2, 6, 559–572.
- P. Peterson. 2009. F2PY: A tool for connecting Fortran and Python programs. *Int. J. Comput. Sci.* 4, 4, 296–305.
- C. W. Rowley. 2005. Model reduction for fluids, using balanced proper orthogonal decomposition. *Int. J. Bifurcat. Chaos* 15, 3, 997–1013.
- C. W. Rowley, I. Mezić, S. Bagheri, P. Schlatter, and D. S. Henningson. 2009. Spectral analysis of nonlinear flows. *J. Fluid Mech.* 641, 115–127.
- P. J. Schmid. 2010. Dynamic mode decomposition of numerical and experimental data. *J. Fluid Mech.* 656, 5–28. DOI: <http://dx.doi.org/10.1017/S0022112010001217>
- L. Sirovich. 1987. Turbulence and the dynamics of coherent structures, parts I–III. *Q. Appl. Math.* 3, 561–590.
- J. H. Tu and C. W. Rowley. 2012. An improved algorithm for balanced POD through an analytic treatment of impulse response tails. *J. Comput. Phys.* 231, 16, 5317–5333.
- J. A. C. Weideman and S. C. Reddy. 2000. A MATLAB differentiation matrix suite. *ACM Trans. Math. Softw.* 26, 4, 465–519.
- K. Willcox and J. Peraire. 2002. Balanced model reduction via the proper orthogonal decomposition. *AIAA J.* 40, 11, 2323–2330.

Received June 2013; revised October 2013; accepted November 2013